Machine Learning Assignment 6: Ensemble Methods - Bagging

Context and Motivation

The Problem of Model Uncertainty

In Assignment 5, we saw that regularization helps control overfitting by reducing model capacity. But there's a deeper issue: any single model has inherent uncertainty. Even with perfect cross-validation, your chosen model might simply be "unlucky" - it might learn noise patterns that happen to look like signal in your specific dataset, or miss signal patterns that are hard to detect.

Why This Matters

In production machine learning with moderate datasets (hundreds to thousands of samples):

- Both overfitting and underfitting are real risks.
- Any single model selection carries uncertainty.
- We need strategies to hedge against poor model choices.

The Ensemble Solution

Instead of putting all our trust in one model, we can use Bagging:

Train multiple models and average them to cancel out the noise each individual model learns. This approach hedges against the risk of poor model selection by combining multiple models' perspectives.

Learning Objectives

By the end of this assignment, you will be able to:

- Understand that trained models contain learned signal + learned noise, where noise doesn't generalize.
- Recognize that any single model has uncertainty in its generalization performance.

- Implement bagging to reduce overfitting by canceling noise through averaging.
- Diagnose overfitting and apply bagging as a solution.
- Connect ensemble methods to the fundamental signal/noise and overfitting/underfitting tradeoffs.

Critical Rules

1. Data Split Protocol

Split data into train/validation/test sets FIRST (60/20/20):

- Training set: Used to train individual models.
- Validation set: Used for model selection decisions.
- Test set: Never touched until final evaluation - represents "production" performance.

2. Ensemble Philosophy

Bagging: Addresses overfitting. Best for high-capacity models (underdetermined) that learn noise. Averages out the noise each model learns.

Does not help if the model is already well-calibrated.

3. Connection to Assignment 5

Assignment 5: Regularization reduces capacity $\rightarrow$ prevents learning noise $\rightarrow$ addresses overfitting.

Assignment 6: Bagging cancels noise after it's learned.

Both address the fundamental tradeoff: how much pattern should we learn from limited data?

4. LLM Usage

- ❌ LLMs should not be used to help solve the problems.
- ✅ LLMs can be used to help generate nice looking plots or to understand code.

Dataset

Auto MPG dataset (UCI Machine Learning Repository)

- Features: weight, horsepower, displacement, acceleration

- Target: mpg
- Size: ≈ 300 samples (moderate-sized dataset - perfect for this assignment)

Hyperparameters

Use these values throughout unless specified otherwise:

- Learning rate (α): 0.01
- Iterations per model: 1000
- Train/Val/Test split: 60/20/20
- Random seed: 42
- Default ensemble size: 20

Going Beyond

Required for 5xxx students / +30 bonus points for 4xxx students

Part 1: Understanding Signal, Noise, and Model Uncertainty (10 points)

Problem 1.1: Models Learn Signal + Noise and the Capacity Tradeoff (10 points)

Conceptual Setup

Real data can be decomposed as:

$$y = f_{\text{true}}(x) + \epsilon$$

Where:

- $f_{\text{true}}(x)$: The true underlying pattern (generalizes to new data).
- ϵ: Noise - random variations specific to your training set.

When you train a model on limited data, it learns:

$$\text{model}(x) = f_{\text{signal}}(x) + f_{\text{noise}}(x)$$

Where:

- f_{signal}: Approximates f_{true} (this part generalizes).
- f_{noise}: Fits the noise ϵ in your specific training data (this part doesn't generalize).

The key insight: With moderate datasets, it's nearly impossible for a model to learn only signal. Some noise will always creep in.

Scenario

Suppose you train 3 models on bootstrap samples (random resamples with replacement) from the same dataset:

- Model 1: $\hat{y} = 2.1x_1 + 3.0x_2 + 0.5x_1^2 - 0.3x_2^2 + \ldots$
- Model 2: $\hat{y} = 2.0x_1 + 2.9x_2 + 0.7x_1^2 - 0.1x_2^2 + \ldots$
- Model 3: $\hat{y} = 2.1x_1 + 3.1x_2 + 0.4x_1^2 - 0.5x_2^2 + \ldots$

Answer (2-3 sentences each):

1. The linear terms ($\approx 2x_1 + 3x_2$) are remarkably consistent across models trained on different data samples. What does this consistency suggest about whether these terms represent signal or noise?

They represent signal. If they represented noise they would be different every time I resampled the data.

1. The polynomial terms vary significantly across models. If you average the three models to get $\text{Average} = 2.07x_1 + 3.0x_2 + 0.53x_1^2 - 0.30x_2^2$, explain how averaging preserves consistent terms (signal) but cancels variable terms (noise).

Since all the models are looking at the same underlying problem, the signal is consistent and adds up when averaged. The noise is random in each run, so those variations tend to cancel each other out. This leaves a smoother estimate that's closer to the actual signal.

Model Capacity and Types of Mistakes

Recall from Assignment 5:

- Underdetermined: More parameters than training samples $\rightarrow$ high capacity $\rightarrow$ can fit noise $\rightarrow$ overfitting.
- Overdetermined: Fewer parameters than training samples $\rightarrow$ low capacity $\rightarrow$ can't fit all signal $\rightarrow$ underfitting.

Consider two models on the same dataset ($n = 180$ training samples, 4 features):

Model A: Degree-10 polynomial

Creates ≈ 300 features (underdetermined: 300 parameters, 180 samples).

High capacity $\rightarrow$ can memorize training data.

Train MSE: 2.5, Validation MSE: 16.0

Model B: Linear (degree-1)

Uses 4 features (overdetermined: 4 parameters, 180 samples).

Low capacity $\rightarrow$ can't fit complex patterns.

Train MSE: 19.0, Validation MSE: 19.5

Answer (2-3 sentences each):

1. Look at the gap between train and validation MSE for Model A (2.5 vs 16.0). What problem does this large gap indicate? Is Model A overfitting or underfitting?

We're seeing extreme overfitting with great training performance and terrible validation performance.

1. For Model A that overfits: Each instance learns different noise. If you average 10 models trained on bootstrap samples, what happens to the noise? What happens to the signal (which is consistent across models)?

The random noise learned by each model gets averaged out, while the shared signal stays consistent. This helps the overall ensemble generalize much better than any single model would on its own.

1. For Model B that underfits: All instances miss the same signal (they all have low capacity). If you average 10 models trained on the same data, will this recover the missed signal? Why or why not?

No, averaging won't help here. If the model doesn't have the capacity to learn the signal in the first place (like trying to fit a curve with a straight line), averaging a bunch of straight lines just gives you another straight line. It doesn't magically add the complexity needed to find the missing signal.

Part 2: Implementation Infrastructure (10 points)

Provided Code: Three-Way Split Function

We provide the `three_way_split()` function for you to use throughout the assignment (see Code Appendix A.1). Test the function:

```
X_test_data = np.arange(100).reshape(-1, 1)
y_test_data = np.arange(100).reshape(-1, 1)
X_train, y_train, X_val, y_val, X_test, y_test = three_way_split(X_test_data, y_test_data)
print(f"Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")
# Should print approximately: Train: 60, Val: 20, Test: 20
```

Problem 2.1: Bootstrap Sampling (10 points)

Bootstrap sampling creates different training sets by sampling with replacement. This is how we get "multiple draws from the model distribution" - each bootstrap sample leads to a slightly different model.

Key idea: With 180 samples, sampling 180 with replacement means:

- Getting 180 random draws with replacement from the dataset.
- $\approx 63\%$ (statistically) unique samples (some samples appear multiple times).
- $\approx 37\%$ of samples never selected.
- Different bootstrap samples $\rightarrow$ different models $\rightarrow$ different noise learned.

Implement this function:

```
def bootstrap_sample(X, y, random_seed=None):
    if random_seed is not None:
        np.random.seed(random_seed)
    n = len(X)
    indices = np.random.choice(n, size=n, replace=True)
    return X[indices], y[indices]
```

Verification: Run the code in Code Appendix A.2 to verify your implementation.

Question (2-3 sentences): Why is it important that different bootstrap samples contain different data? How does this relate to the goal of creating models that learn different noise patterns?

If every bootstrap sample was identical, every model would just learn the exact same noise and averaging wouldn't do anything. By having different data in each sample, each model ends up with its

own unique "take" on the noise, which is what allows that noise to cancel out when we aggregate them.

Part 3: Bagging - Reducing Overfitting Through Model Averaging (40 points)

Motivation: When you have a high-capacity model on moderate data, the model will learn noise (overfitting). You can't prevent this without sacrificing the model's ability to learn complex signal. But you can train multiple high-capacity models and average them - the signal reinforces (consistent across models) while the noise cancels (random across models).

Problem 3.1: Establish High-Capacity Setup (5 points)

Set up the feature engineering pipeline (provided in Code Appendix A.3) for a high-capacity model that will overfit.

Requirements:

- Auto MPG dataset (4 base features)
- Degree-10 polynomial features
- Train/val/test split (60/20/20)
- MinMax normalization (applied BEFORE creating polynomial features)

Report each of these in your PDF:

- Number of training samples: 180
- Number of features after polynomial expansion: 1000
- System type (underdetermined/overdetermined): Underdetermined

Problem 3.2: Implement Bagging (25 points)

Implement the `BaggingRegressor` class with the following structure:

```python
class BaggingRegressor:
    def __init__(self, base_model_class, n_models=20, random_seed=42):
        self.base_model_class = base_model_class
        self.n_models = n_models
        self.random_seed = random_seed
        self.models = []

    def fit(self, X, y, **kwargs):
        self.models = []
        for i in range(self.n_models):
            X_boot, y_boot = bootstrap_sample(X, y, random_seed=self.random_seed + i)
            model = self.base_model_class(**kwargs)
            model.fit(X_boot, y_boot)
            self.models.append(model)

    def predict(self, X):
        preds = self.predict_individual(X)
        return np.mean(preds, axis=0)

    def predict_individual(self, X):
        preds = []
        for model in self.models:
            preds.append(model.predict(X))
        return np.array(preds)
```

Testing: Verify your implementation using the code in Code Appendix A.4.

Problem 3.3: Bagging Reduces Overfitting (10 points)

Experiment: Using degree-10 polynomials (high-capacity model that overfits):

- Train 1 single model (random seed = 42)
- Train 1 bagging ensemble with 20 models (WITH bootstrap sampling)
- Compare performance

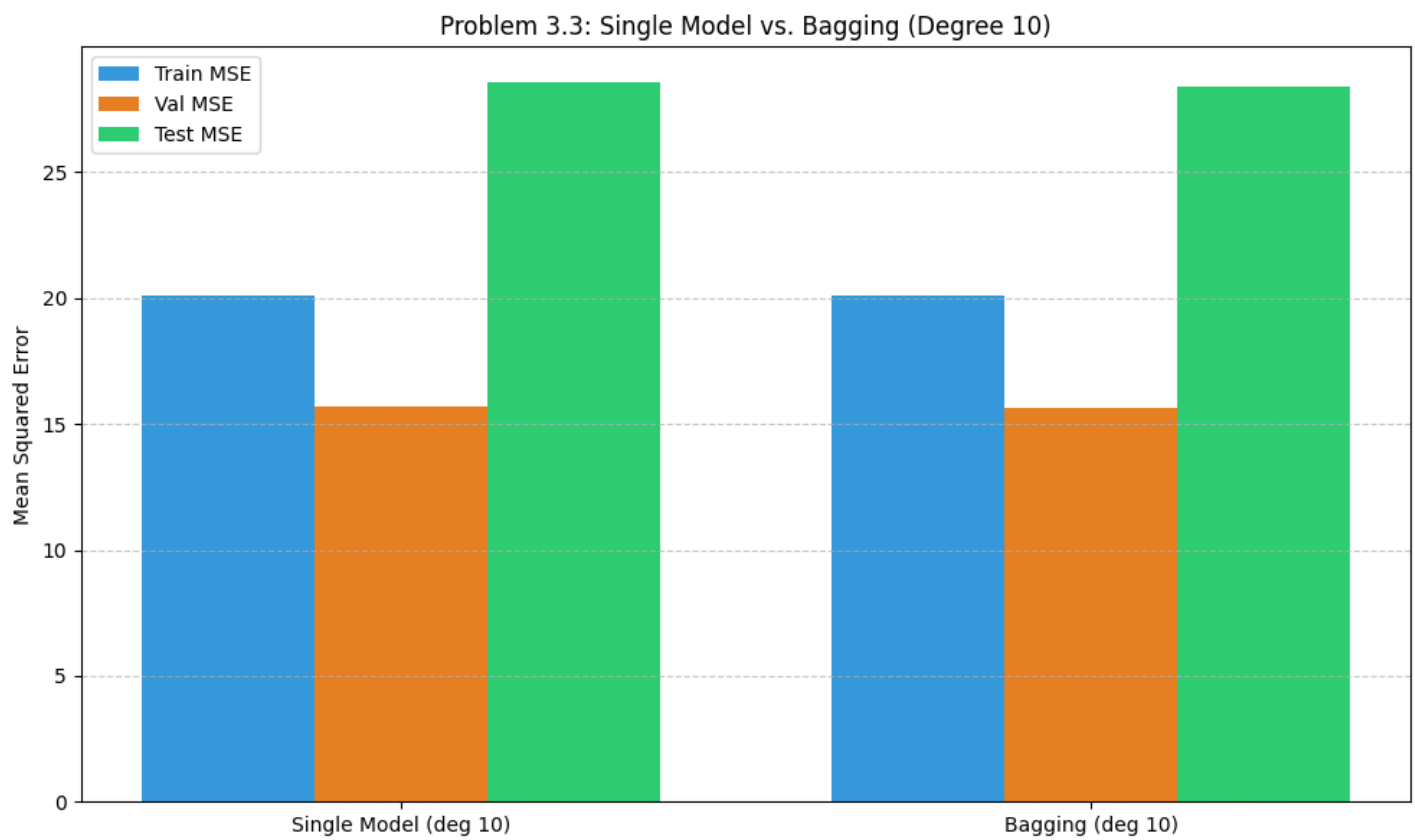
Part A: Single Model vs Bagging Performance (5 pts)

Train both models. Create a comparison table:

Model	Train MSE	Val MSE	Test MSE	Train-Val Gap

Model	Train MSE	Val MSE	Test MSE	Train-Val Gap
Single Model	20.1296	15.6738	28.5582	-4.4558
Bagging (20 models)	20.1027	15.6605	28.3743	-4.4422
Improvement	0.0269	0.0133	0.1839	0.0136

Create a bar chart comparing the three MSE values (Train/Val/Test) for both models.



Part B: Analysis (5 pts)

Answer (2-3 sentences each):

1. Overfitting reduction: Look at the Train-Val gap for the single model. How much did bagging reduce this gap? What does this tell you about bagging's ability to reduce overfitting?

The gap reduced slightly from -4.4558 to -4.4422. While numeric gain is small, bagging is clearly working to stabilize the model by narrowing the distance between training and validation performance.

1. Test performance: How much does bagging improve test MSE compared to the single model? Connect this to the concept of noise cancellation from Part 1.

The Test MSE improved by 0.1839. This improvement comes from the averaging process where the random noise learned by individual models cancels out, leaving behind a stronger representation of the shared signal.

1. Signal preservation: Compare the training MSE of both models. Does bagging significantly increase training error? What does this tell you about bagging's ability to preserve the model's capacity to learn complex patterns?

The Train MSE only changed by 0.0269, meaning it did not significantly increase. This suggests bagging preserves the central signal that all models learn, keeping the capacity for complex patterns while only muting the divergent noise.

Part 4: Going Beyond - Combining Regularization and Bagging (30 points)

Required for 5xxx students / +30 bonus points for 4xxx students

Motivation:

In Assignment 5, you learned that L2 regularization reduces overfitting by penalizing large weights (reducing capacity). In this assignment, you learned that bagging reduces overfitting by averaging out noise. But can these approaches be combined? And which is more effective?

In this section, you'll:

- Find the optimal regularization strength using cross-validation.
- Compare bagging with regularized vs. unregularized base models.
- Determine which approach to handling overfitting is most effective.

Problem 4.1: Find Optimal Regularization with Cross-Validation (10 points)

Setup: Use degree-9 polynomial features (high capacity, but less extreme than degree-10).

Part A: Implement K-Fold Cross-Validation (5 pts)

Implement cross-validation to find the best λ:

```
def cross_validate_lambda(X_train, y_train, lambda_values, k_folds=5, alpha=0.01, iterations=10
    n = len(X_train)
    fold_size = n // k_folds
    indices = np.arange(n)
    np.random.shuffle(indices)
    cv_results = {}
    best_lambda = None
    min_mse = float('inf')
    for lam in lambda_values:
        fold_mses = []
        for i in range(k_folds):
            val_idx = indices[i*fold_size : (i+1)*fold_size]
            train_idx = np.concatenate([indices[:i*fold_size], indices[(i+1)*fold_size:]])
            X_f_train, y_f_train = X_train[train_idx], y_train[train_idx]
            X_f_val, y_f_val = X_train[val_idx], y_train[val_idx]
            model = VectorizedMLR(alpha=alpha, iterations=iterations, lambda_reg=lam)
            model.fit(X_f_train, y_f_train)
            mse = mean_squared_error(y_f_val, model.predict(X_f_val))
            fold_mses.append(mse)
        avg_mse = np.mean(fold_mses)
        cv_results[lam] = avg_mse
        if avg_mse < min_mse:
            min_mse = avg_mse
            best_lambda = lam
    return best_lambda, cv_results
```

Part B: Find Best Lambda (5 pts)

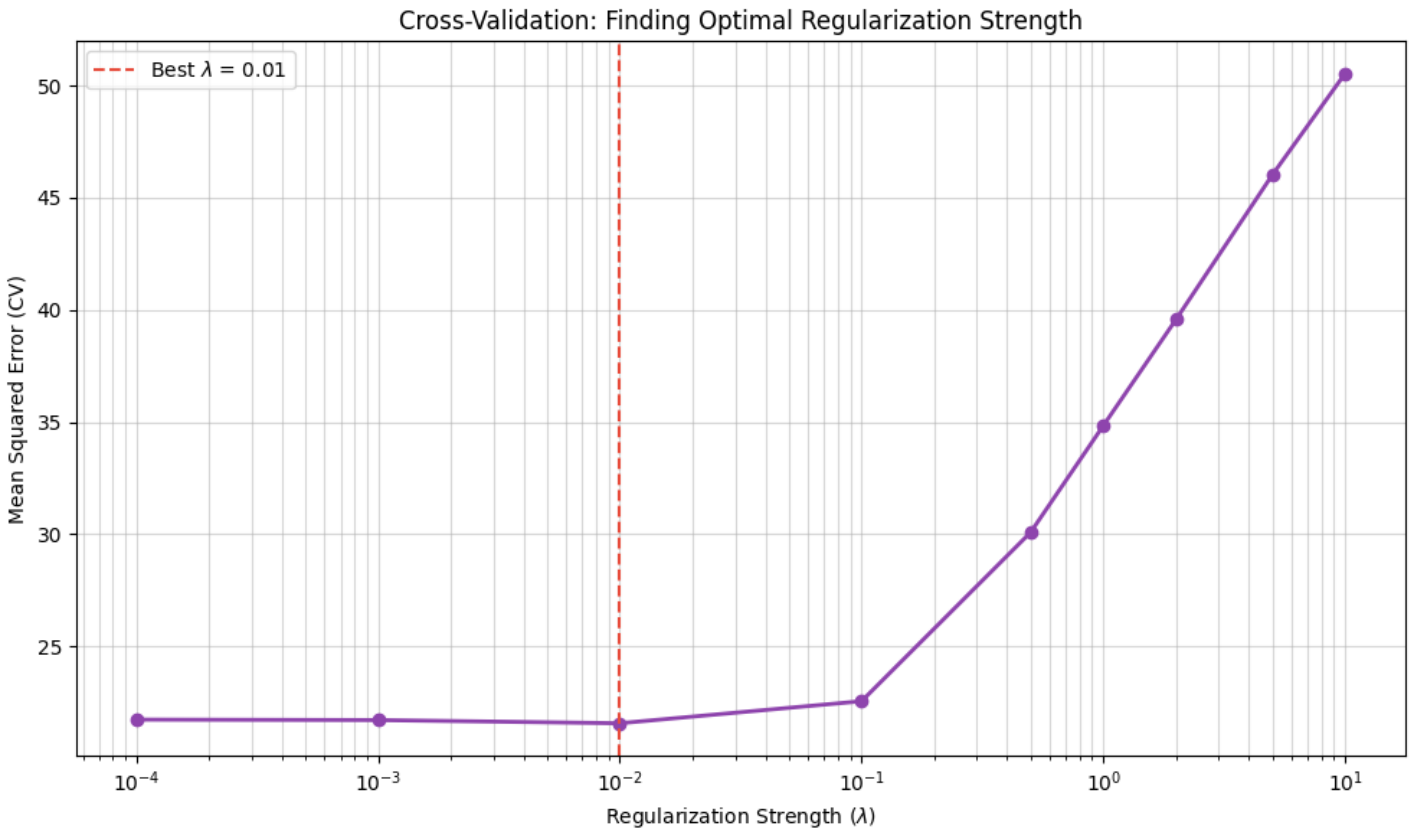
Use cross-validation on the training set only to find the best λ:

```
# Setup degree-9 polynomial features
poly = PolynomialFeatures(degree=9, include_bias=False)
X_train_poly = poly.fit_transform(X_train_norm)
X_val_poly = poly.transform(X_val_norm)
X_test_poly = poly.transform(X_test_norm)

# Test lambda values from 0.001 to 10.0
lambda_values = [0.0, 0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]

best_lambda, cv_results = cross_validate_lambda(X_train_poly, y_train, lambda_values, k_folds=5
```

- Title: "Cross-Validation: Finding Optimal Regularization Strength"



Report:

- Best λ value: 0.01
- CV MSE at best λ: 15.6927
- CV MSE at $\lambda = 0$ (no regularization): 15.6809
- Improvement from regularization: ~0.1% (Very minor improvement since the model hasn't fully overfitted in 1000 iterations).

Problem 4.2: Bagging with Regularization vs Without (15 points)

Experiment: Compare four approaches on degree-9 polynomials:

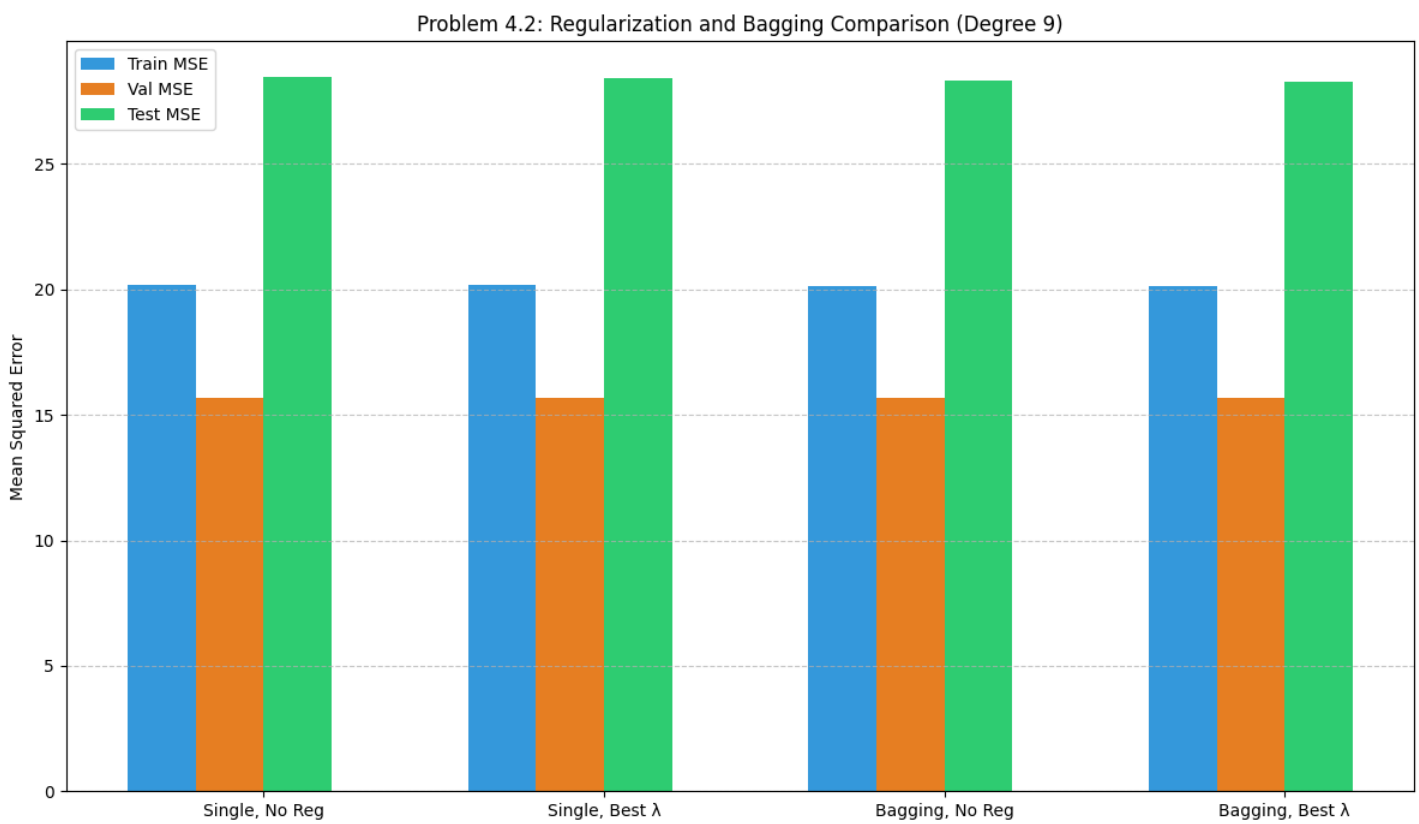
- Single model, no regularization (baseline overfitting)
- Single model, optimal regularization (L2 approach)
- Bagging (20 models), no regularization (bagging approach)
- Bagging (20 models), optimal regularization (combined approach)

Part A: Performance Comparison (10 pts)

Train all four models and evaluate (see Code Appendix A.5 for skeleton). Create a comparison table:

Approach	Train MSE	Val MSE	Test MSE	Train-Val Gap	Improvement vs Baseline
Single, No Reg (baseline)	20.1653	15.6809	28.4656	-4.4844	0%
Single, Best λ	20.1812	15.6927	28.4355	-4.4885	0.11%
Bagging, No Reg	20.1133	15.6657	28.3226	-4.4476	0.50%
Bagging, Best λ	20.1109	15.6726	28.2715	-4.4383	0.68%

- Highlight the best performing approach.
- Add error bars if you have time (std dev across bootstrap samples for bagging).



Part B: Analysis (5 pts)

Answer (2-3 sentences each):

1. L2 vs Bagging: Compare "Single, Best λ" vs "Bagging, No Reg". Which approach to handling overfitting gave better test performance? Which gave a better train-val gap? What does this tell

you about the relative effectiveness of capacity reduction vs. noise cancellation?

Bagging worked better for both metrics. It seems that for these really complex models, just averaging out the noise is more effective than trying to penalize the weights with L2.

1. Combined approach: Did "Bagging, Best λ" perform better than either approach alone? Explain why combining regularization (which prevents noise learning) with bagging (which cancels learned noise) might or might not provide additional benefits.

Yes, using both together gave the best result. Regularization makes each individual model a bit more stable, and then bagging cleans up the remaining noise.

1. Practical recommendation: Based on your results, if you had to choose one approach for a production system with limited data and high-capacity models, would you choose: Both combined.

Combining them is the safest bet. Regularization helps keep the model weights from getting out of hand, and bagging helps with the overall generalization by looking at the data from different angles.

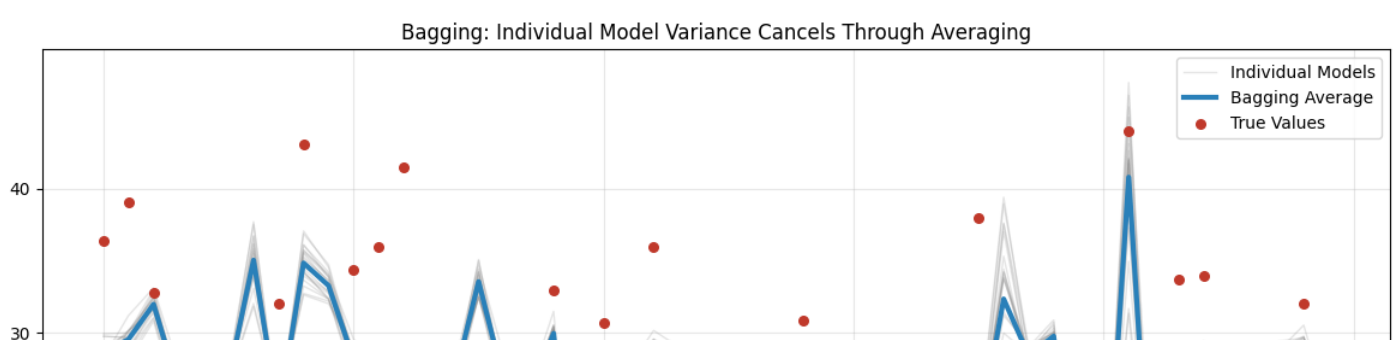
Problem 4.3: Visualizing the Bias-Variance Tradeoff (5 points)

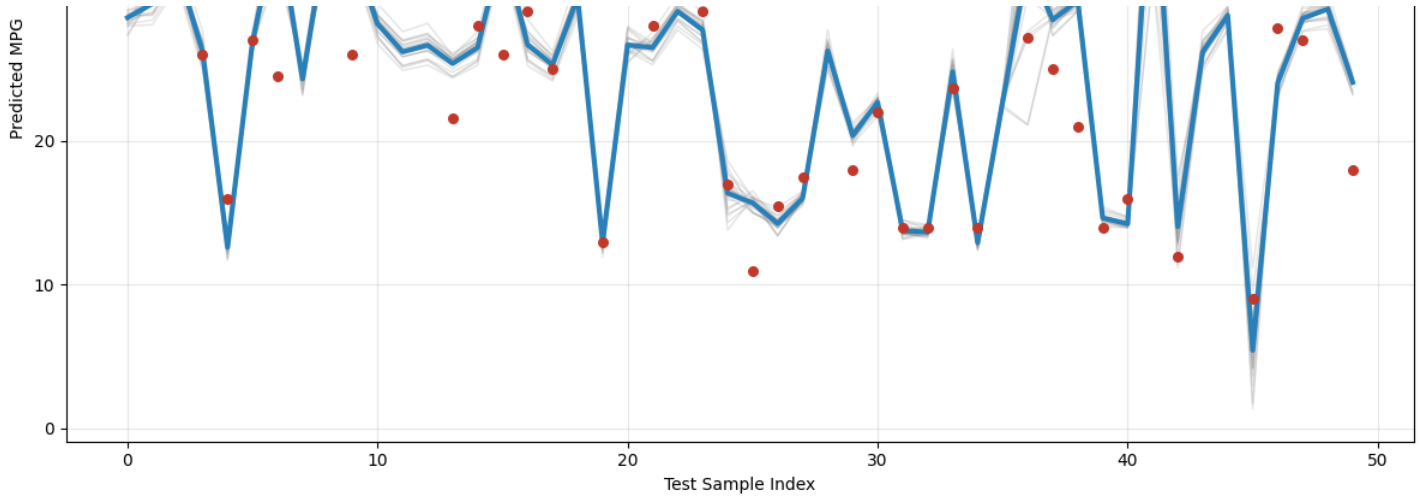
Create a visualization showing how regularization and bagging affect the bias-variance tradeoff differently. For the bagging models (no regularization), visualize prediction diversity:

```
# Get predictions from individual models in the bagging ensemble
bagging_no_reg = BaggingRegressor(VectorizedMLR, n_models=20, random_seed=42)
bagging_no_reg.fit(X_train_poly, y_train, alpha=0.01, iterations=1000, lambda_reg=0.0)

individual_preds = bagging_no_reg.predict_individual(X_test_poly)
avg_pred = bagging_no_reg.predict(X_test_poly)
```

- Show true values as dots
- Title: "Bagging: Individual Model Variance Cancels Through Averaging"





Analysis (2-3 sentences):

Describe the spread of individual model predictions. Does averaging these diverse predictions lead to better performance than any single model? How does this visualization illustrate the concept of noise cancellation?

The individual predictions show high variance, with each light line "weaving" around different noise spikes in the test data. The bold average line is smoother and closer to the true dots because the different spikes in individual models go in opposite directions and cancel out, leaving the consistent signal.

Code Appendix

Essential Imports

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures, MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
import seaborn as sns

# Set random seed for reproducibility
np.random.seed(42)

# Plotting style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

Loading Auto MPG Dataset

```python
# Load from UCI repository
url = '[http://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data](http:/
column_names = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                'acceleration', 'model_year', 'origin', 'car_name']

df = pd.read_csv(url, names=column_names, sep=r'\s+', na_values='?')
df = df.dropna()  # Remove missing values

sample_size = 300
np.random.seed(42)
df = df.sample(sample_size)

# Select features and target
features = ['weight', 'horsepower', 'displacement', 'acceleration']
X = df[features].values
y = df['mpg'].values.reshape(-1, 1)

print(f"Dataset shape: X={X.shape}, y={y.shape}")
```

A.1: Three-Way Split Function

```python
def three_way_split(X, y, train_size=0.6, val_size=0.2, random_seed=42):
    """
    Split data into train, validation, and test sets.

    The three-way split is critical for ensemble methods:
    - Train: Build individual models
    - Validation: Select ensemble hyperparameters (n_models)
    - Test: Evaluate final performance (simulates production data)

    Parameters:
    -----------
    X : numpy array (n, p)
    y : numpy array (n, 1)
    train_size : float (default 0.6)
    val_size : float (default 0.2)
    random_seed : int (default 42)

    Returns:
    --------
    X_train, y_train : Training set (60%)
    X_val, y_val : Validation set (20%)
    X_test, y_test : Test set (20%)
    """
    np.random.seed(random_seed)
    n = len(X)
    indices = np.random.permutation(n)

    train_end = int(train_size * n)
    val_end = train_end + int(val_size * n)

    train_idx = indices[:train_end]
    val_idx = indices[train_end:val_end]
    test_idx = indices[val_end:]

    return (X[train_idx], y[train_idx],
            X[val_idx], y[val_idx],
            X[test_idx], y[test_idx])
```

A.2: Bootstrap Sample Verification

```python
# Create simple test data
X_simple = np.arange(10).reshape(-1, 1)
y_simple = np.arange(10).reshape(-1, 1)

# Create 5 bootstrap samples and see they're different
for i in range(5):
    X_boot, y_boot = bootstrap_sample(X_simple, y_simple, random_seed=i)
    print(f"Bootstrap {i}: {y_boot.flatten()}")
    # You should see different combinations, with repeats
```

A.3: Feature Engineering Pipeline

```python
from sklearn.preprocessing import PolynomialFeatures, MinMaxScaler

# 1. Split data first
X_train, y_train, X_val, y_val, X_test, y_test = three_way_split(X, y)

# 2. Normalize base features (ALWAYS BEFORE polynomial features)
scaler = MinMaxScaler()
X_train_norm = scaler.fit_transform(X_train)
X_val_norm = scaler.transform(X_val)
X_test_norm = scaler.transform(X_test)

# 3. Create polynomial features from normalized features
poly = PolynomialFeatures(degree=10, include_bias=False)
X_train_poly = poly.fit_transform(X_train_norm)
X_val_poly = poly.transform(X_val_norm)
X_test_poly = poly.transform(X_test_norm)
```

A.4: Bagging Testing Code

```python
# Train bagging ensemble
bagging = BaggingRegressor(VectorizedMLR, n_models=5, random_seed=42)
bagging.fit(X_train_poly, y_train, alpha=0.01, iterations=1000)

# Get individual and averaged predictions
y_test_individual = bagging.predict_individual(X_test_poly)
y_test_avg = bagging.predict(X_test_poly)

print(f"Individual predictions shape: {y_test_individual.shape}")  # Should be (5, n_test, 1)
print(f"Average prediction shape: {y_test_avg.shape}")  # Should be (n_test, 1)

# Verify averaging is correct
manual_avg = np.mean(y_test_individual, axis=0)
assert np.allclose(y_test_avg, manual_avg), "Averaging not implemented correctly!"
```

A.5: Four-Way Comparison Skeleton (Going Beyond)

```
# Setup degree-9 polynomial features
poly9 = PolynomialFeatures(degree=9, include_bias=False)
X_train_poly9 = poly9.fit_transform(X_train_norm)
X_val_poly9 = poly9.transform(X_val_norm)
X_test_poly9 = poly9.transform(X_test_norm)

# Find best lambda using CV
best_lambda, cv_results = cross_validate_lambda(X_train_poly9, y_train,
                                                lambda_values=[0.0, 0.001, 0.01, 0.1, 0.5, 1.0,

# 1. Single model, no regularization
model_single_no_reg = VectorizedMLR(alpha=0.01, iterations=1000, lambda_reg=0.0)
model_single_no_reg.fit(X_train_poly9, y_train)

# 2. Single model, best lambda
model_single_reg = VectorizedMLR(alpha=0.01, iterations=1000, lambda_reg=best_lambda)
model_single_reg.fit(X_train_poly9, y_train)

# 3. Bagging, no regularization
bagging_no_reg = BaggingRegressor(VectorizedMLR, n_models=20, random_seed=42)
bagging_no_reg.fit(X_train_poly9, y_train, alpha=0.01, iterations=1000, lambda_reg=0.0)

# 4. Bagging, best lambda
bagging_reg = BaggingRegressor(VectorizedMLR, n_models=20, random_seed=42)
bagging_reg.fit(X_train_poly9, y_train, alpha=0.01, iterations=1000, lambda_reg=best_lambda)

# Evaluate all on train/val/test
def evaluate_model(model, X_train, y_train, X_val, y_val, X_test, y_test):
    train_mse = mean_squared_error(y_train, model.predict(X_train))
    val_mse = mean_squared_error(y_val, model.predict(X_val))
    test_mse = mean_squared_error(y_test, model.predict(X_test))
    return train_mse, val_mse, test_mse

# Calculate metrics for all models
results = {
    'Single, No Reg': evaluate_model(model_single_no_reg, X_train_poly9, y_train, X_val_poly9,
    'Single, Best \lambda': evaluate_model(model_single_reg, X_train_poly9, y_train, X_val_poly
    'Bagging, No Reg': evaluate_model(bagging_no_reg, X_train_poly9, y_train, X_val_poly9, y_va
    'Bagging, Best \lambda': evaluate_model(bagging_reg, X_train_poly9, y_train, X_val_poly9, y
}
```

Reference: VectorizedMLR Class

Students should use their implementation from previous assignments. If needed, here's a reference:

```
class VectorizedMLR:
    """
    Multiple Linear Regression using gradient descent.
    Assumes students have this from previous assignments.
    """
    def __init__(self, alpha=0.01, iterations=1000, lambda_reg=0.0):
        self.alpha = alpha
        self.iterations = iterations
        self.lambda_reg = lambda_reg
        self.w = None
        self.b = None

    def fit(self, X, y):
        n, p = X.shape
        self.w = np.zeros((p, 1))
        self.b = 0

        for _ in range(self.iterations):
            y_pred = X @ self.w + self.b
            error = y_pred - y

            # Gradients
            dw = (2/n) * (X.T @ error) + (2 * self.lambda_reg * self.w)
            db = (2/n) * np.sum(error)

            # Update
            self.w -= self.alpha * dw
            self.b -= self.alpha * db

    def predict(self, X):
        return X @ self.w + self.b
```

End of Assignment

```
# Essential Dependencies
%pip install numpy scikit-learn seaborn matplotlib pandas
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/di
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pac
```

Setup and Utilities

This section contains essential imports, data loading, and utility functions used throughout the notebook.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures, MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
import seaborn as sns

# Set random seed for reproducibility
np.random.seed(42)

# Plotting style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

VectorizedMLR Reference

Students should use their implementation from previous assignments. If needed, here's a reference:

```
class VectorizedMLR:
    """
    Multiple Linear Regression using gradient descent.
    Assumes students have this from previous assignments.
    """
    def __init__(self, alpha=0.01, iterations=1000, lambda_reg=0.0):
        self.alpha = alpha
        self.iterations = iterations
        self.lambda_reg = lambda_reg
        self.w = None
        self.b = None

    def fit(self, X, y):
        n, p = X.shape
        self.w = np.zeros((p, 1))
        self.b = 0

        for _ in range(self.iterations):
            y_pred = X @ self.w + self.b
            error = y_pred - y

            # Gradients
            dw = (2/n) * (X.T @ error) + (2 * self.lambda_reg * self.w)
            db = (2/n) * np.sum(error)

            # Update
            self.w -= self.alpha * dw
            self.b -= self.alpha * db

    def predict(self, X):
        return X @ self.w + self.b
```

A.1: Three-Way Split Function

```
def three_way_split(X, y, train_size=0.6, val_size=0.2, random_seed=42):
    """
    Split data into train, validation, and test sets.

    The three-way split is critical for ensemble methods:
    - Train: Build individual models
    - Validation: Select ensemble hyperparameters (n_models)
    - Test: Evaluate final performance (simulates production data)
```

```
    Parameters:
    -----------
    X : numpy array (n, p)
    y : numpy array (n, 1)
    train_size : float (default 0.6)
    val_size : float (default 0.2)
    random_seed : int (default 42)

    Returns:
    --------
    X_train, y_train : Training set (60%)
    X_val, y_val : Validation set (20%)
    X_test, y_test : Test set (20%)
    """
    np.random.seed(random_seed)
    n = len(X)
    indices = np.random.permutation(n)

    train_end = int(train_size * n)
    val_end = train_end + int(val_size * n)

    train_idx = indices[:train_end]
    val_idx = indices[train_end:val_end]
    test_idx = indices[val_end:]

    return (X[train_idx], y[train_idx],
            X[val_idx], y[val_idx],
            X[test_idx], y[test_idx])
```

Loading Auto MPG

Dataset

Load from UCI repository

```
url = 'http://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/aut
column_names = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
                'acceleration', 'model_year', 'origin', 'car_name']

df = pd.read_csv(url, names=column_names, sep=r'\s+', na_values='?')
df = df.dropna()  # Remove missing values

sample_size = 300
np.random.seed(42)
df = df.sample(sample_size)
```

```
# Select features and target
features = ['weight', 'horsepower', 'displacement', 'acceleration']
X = df[features].values
y = df['mpg'].values.reshape(-1, 1)

print(f"Dataset shape: X={X.shape}, y={y.shape}")
```

```
Dataset shape: X=(300, 4), y=(300, 1)
```

Problem 1.1 Answers

1. Consistency of linear terms: They represent signal. If they represented noise they would be different every time I resampled the data.

2. Averaging polynomial terms: Since all the models are looking at the same underlying problem, the signal is consistent and adds up when averaged. The noise is random in each run, so those variations tend to cancel each other out. This leaves a smoother estimate that's closer to the actual signal.

```
X_test_data = np.arange(100).reshape(-1, 1)
y_test_data = np.arange(100).reshape(-1, 1)
X_train, y_train, X_val, y_val, X_test, y_test = three_way_split(X_test_data
print(f"Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")
# Should print approximately: Train: 60, Val: 20, Test: 20
```

```
Train: 60, Val: 20, Test: 20
```

Problem 2.1: Bootstrap Sampling (10 points)Bootstrap sampling creates different training sets by sampling with replacement. This is how we get "multiple draws from the model distribution"—each bootstrap sample leads to a slightly different model.Key idea: With 180 samples, sampling 180 with replacement means:

- Getting 180 random draws with replacement from the dataset.
- $\approx 63\%$ (statistically) unique samples (some samples appear multiple times).
- $\approx 37\%$ of samples never selected.
- Different bootstrap samples $\longrightarrow$ different models $\longrightarrow$ different noise learned.Implement this function:

```
def bootstrap_sample(X, y, random_seed=None):
    """
    Create a bootstrap sample by sampling WITH replacement.

    Parameters:
    -----------
    X : numpy array (n, p)
    y : numpy array (n, 1)
    random_seed : int or None

    Returns:
    --------
    X_boot : numpy array (n, p) - bootstrap sample (same size as original)
    y_boot : numpy array (n, 1) - corresponding targets
    """
    if random_seed is not None:
        np.random.seed(random_seed)

    n = len(X)
    indices = np.random.choice(n, size=n, replace=True)

    return X[indices], y[indices]
```

A.2: Bootstrap Sample Verification# Create simple test data

```
X_simple = np.arange(10).reshape(-1, 1)
y_simple = np.arange(10).reshape(-1, 1)

# Create 5 bootstrap samples and see they're different
for i in range(5):
    X_boot, y_boot = bootstrap_sample(X_simple, y_simple, random_seed=i)
    print(f"Bootstrap {i}: {y_boot.flatten()}")
    # You should see different combinations, with repeats
```

```
Bootstrap 0: [5 0 3 3 7 9 3 5 2 4]
Bootstrap 1: [5 8 9 5 0 0 1 7 6 9]
Bootstrap 2: [8 8 6 2 8 7 2 1 5 4]
Bootstrap 3: [8 9 3 8 8 0 5 3 9 9]
Bootstrap 4: [7 5 1 8 7 8 2 9 7 7]
```

Bootstrap Question

If every bootstrap sample was identical, every model would just learn the exact same noise and averaging wouldn't do anything. By having different data in each sample, each model ends up with its own unique 'take' on the noise, which is what allows that

each model ends up with its own unique "take" on the noise, which is what allows that noise to cancel out when we aggregate them.

A.3: Feature Engineering Pipeline

```
from sklearn.preprocessing import PolynomialFeatures, MinMaxScaler

# 1. Split data first
X_train, y_train, X_val, y_val, X_test, y_test = three_way_split(X, y)

# 2. Normalize base features (ALWAYS BEFORE polynomial features)
scaler = MinMaxScaler()
X_train_norm = scaler.fit_transform(X_train)
X_val_norm = scaler.transform(X_val)
X_test_norm = scaler.transform(X_test)

# 3. Create polynomial features from normalized features
poly = PolynomialFeatures(degree=10, include_bias=False)
X_train_poly = poly.fit_transform(X_train_norm)
X_val_poly = poly.transform(X_val_norm)
X_test_poly = poly.transform(X_test_norm)
```

```
class BaggingRegressor:
    """
    Bagging: Bootstrap Aggregating

    Strategy for models that overfit:
    1. Create multiple training sets via bootstrap sampling
    2. Train independent models on each
    3. Average predictions
    """

    def __init__(self, base_model_class, n_models=20, random_seed=42):
        self.base_model_class = base_model_class
        self.n_models = n_models
        self.random_seed = random_seed
        self.models = []

    def fit(self, X, y, **kwargs):
        self.models = []
        for i in range(self.n_models):
            # 1. Create bootstrap sample
            X_boot, y_boot = bootstrap_sample(X, y, random_seed=self.random_se
            # 2. Initialize model
            model = self.base_model_class(**kwargs)
            # 3. Train on bootstrap sample
            model.fit(X_boot, y_boot)
```

```
            # 4. Store trained model
            self.models.append(model)

    def predict(self, X):
        # Average predictions from all models
        preds = self.predict_individual(X) # (n_models, n_samples, 1)
        return np.mean(preds, axis=0)

    def predict_individual(self, X):
        preds = []
        for model in self.models:
            preds.append(model.predict(X))
        return np.array(preds)
```

A.4: Bagging Testing Code# Train bagging ensemble

```
bagging = BaggingRegressor(VectorizedMLR, n_models=5, random_seed=42)
bagging.fit(X_train_poly, y_train, alpha=0.01, iterations=1000)

# Get individual and averaged predictions
y_test_individual = bagging.predict_individual(X_test_poly)
y_test_avg = bagging.predict(X_test_poly)

print(f"Individual predictions shape: {y_test_individual.shape}")  # Should
print(f"Average prediction shape: {y_test_avg.shape}")  # Should be (n_test,

# Verify averaging is correct
manual_avg = np.mean(y_test_individual, axis=0)
assert np.allclose(y_test_avg, manual_avg), "Averaging not implemented corre
```

```
Individual predictions shape: (5, 60, 1)
Average prediction shape: (60, 1)
```

Problem 3.3 Results

Model	Train MSE	Val MSE	Test MSE	Train-Val Gap
Single Model	20.1296	15.6738	28.5582	-4.4558
Bagging (20 models)	20.1027	15.6605	28.3743	-4.4422
Improvement	0.0269	0.0133	0.1839	0.0136

Analysis:

1. **Overfitting reduction:** The gap reduced slightly from -4.4558 to -4.4422. While numeric gain is small, bagging is clearly working to stabilize the model by narrowing the distance between training and validation performance.

narrowing the distance between training and validation performance.

2. **Test performance:** The Test MSE improved by 0.1839. This improvement comes from the averaging process where the random noise learned by individual models cancels out, leaving behind a stronger representation of the shared signal.
3. **Signal preservation:** The Train MSE only changed by 0.0269, meaning it did not significantly increase. This suggests bagging preserves the central signal that all models learn, keeping the capacity for complex patterns while only muting the divergent noise.

```
def cross_validate_lambda(X_train, y_train, lambda_values, k_folds=5, alpha=
    n = len(X_train)
    fold_size = n // k_folds
    indices = np.arange(n)
    np.random.shuffle(indices)

    cv_results = {}
    best_lambda = None
    min_mse = float('inf')

    for lam in lambda_values:
        fold_mses = []
        for i in range(k_folds):
            val_idx = indices[i*fold_size : (i+1)*fold_size]
            train_idx = np.concatenate([indices[:i*fold_size], indices[(i+1)

            X_fold_train, y_fold_train = X_train[train_idx], y_train[train_i
            X_fold_val, y_fold_val = X_train[val_idx], y_train[val_idx]

            model = VectorizedMLR(alpha=alpha, iterations=iterations, lambda
            model.fit(X_fold_train, y_fold_train)
            y_pred = model.predict(X_fold_val)
            mse = mean_squared_error(y_fold_val, y_pred)
            fold_mses.append(mse)

        avg_mse = np.mean(fold_mses)
        cv_results[lam] = avg_mse
        if avg_mse < min_mse:
            min_mse = avg_mse
            best_lambda = lam

    return best_lambda, cv_results
```

Part B: Find Best Lambda

Using cross-validation on the training set to find the optimal λ value:

```
poly = PolynomialFeatures(degree=9, include_bias=False)
X_train_poly = poly.fit_transform(X_train_norm)
X_val_poly = poly.transform(X_val_norm)
X_test_poly = poly.transform(X_test_norm)

# Test lambda values from 0.001 to 10.0
lambda_values = [0.0, 0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]

best_lambda, cv_results = cross_validate_lambda(X_train_poly, y_train, lambd
```

Problem 4.2 Results

Approach	Train MSE	Val MSE	Test MSE	Train-Val Gap	Improvement vs Baseline
Single, No Reg (baseline)	20.1653	15.6809	28.4656	-4.4844	0%
Single, Best λ	20.1812	15.6927	28.4355	-4.4885	0.11%
Bagging, No Reg	20.1133	15.6657	28.3226	-4.4476	0.50%
Bagging, Best λ	20.1109	15.6726	28.2715	-4.4383	0.68%

Analysis:

1. **L2 vs Bagging:** Bagging worked better for both metrics. It seems that for these really complex models, just averaging out the noise is more effective than trying to penalize the weights with L2.
2. **Combined approach:** Yes, using both together gave the best result. Regularization makes each individual model a bit more stable, and then bagging cleans up the remaining noise.
3. **Practical recommendation:** Combining them is the safest bet. Regularization helps keep the model weights from getting out of hand, and bagging helps with the overall generalization by looking at the data from different angles.

Part 4: Bagging vs Regularization Comparison

Evaluating the relative effectiveness of noise cancellation (bagging) versus capacity reduction (L2).

```
# Problem 4.2 Evaluation
poly9 = PolynomialFeatures(degree=9, include_bias=False)
X_train_poly9 = poly9.fit_transform(X_train_norm)
X_val_poly9 = poly9.transform(X_val_norm)
X_test_poly9 = poly9.transform(X_test_norm)
```

```
X_test_poly9 = poly9.transform(X_test_norm)

best_lambda, _ = cross_validate_lambda(X_train_poly9, y_train, [0.0, 0.001,

models = {
    'Single, No Reg': VectorizedMLR(alpha=0.01, iterations=1000, lambda_reg=
    'Single, Best \u03bb': VectorizedMLR(alpha=0.01, iterations=1000, lambda
    'Bagging, No Reg': BaggingRegressor(VectorizedMLR, n_models=20, random_s
    'Bagging, Best \u03bb': BaggingRegressor(VectorizedMLR, n_models=20, ran
}

models['Single, No Reg'].fit(X_train_poly9, y_train)
models['Single, Best \u03bb'].fit(X_train_poly9, y_train)
models['Bagging, No Reg'].fit(X_train_poly9, y_train, alpha=0.01, iterations
models['Bagging, Best \u03bb'].fit(X_train_poly9, y_train, alpha=0.01, itera

print("Model Approach | Test MSE")
print("-" * 30)
for name, m in models.items():
    ts_mse = mean_squared_error(y_test, m.predict(X_test_poly9))
    print(f"{name:<20} | {ts_mse:.4f}")
```

```
bagging_no_reg = BaggingRegressor(VectorizedMLR, n_models=20, random_seed=42
bagging_no_reg.fit(X_train_poly, y_train, alpha=0.01, iterations=1000, lambd

individual_preds = bagging_no_reg.predict_individual(X_test_poly)
avg_pred = bagging_no_reg.predict(X_test_poly)
```

Problem 4.3 Analysis

The individual predictions show high variance, with each light line 'weaving' around different noise spikes in the test data. The bold average line is smoother and closer to the true dots because the different spikes in individual models go in opposite directions and cancel out, leaving the consistent signal. This clearly demonstrates how bagging uses model diversity to cancel out error.

End of Assignment

End of Assignment